

CS 255: Computer Architecture and Machine Level Programming

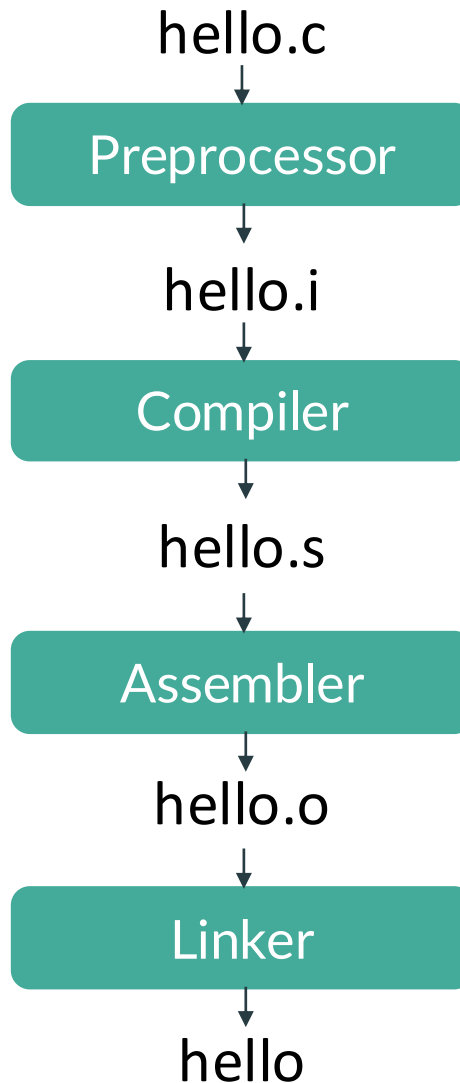
Prof. Martha Barker

C Compilation and Debugging

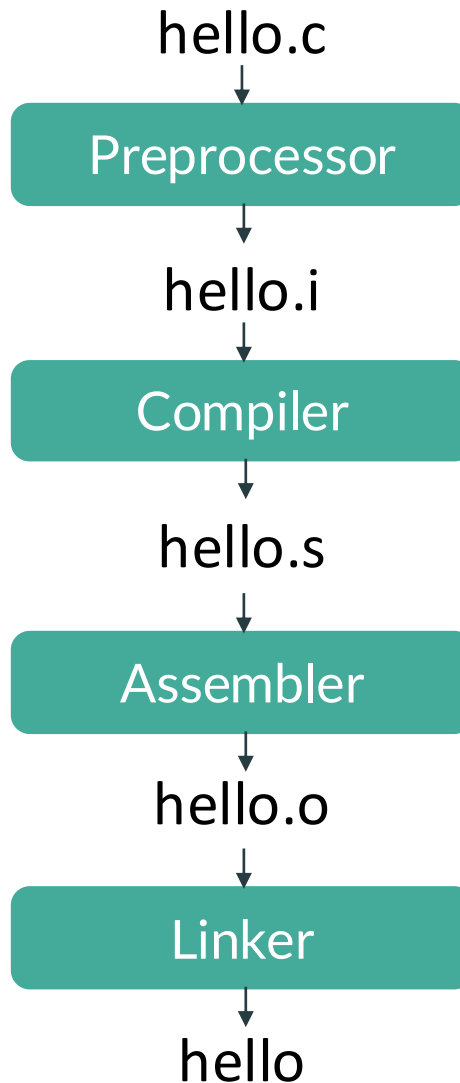
Compilation

- Compilation steps
- (More) gcc

Compilation is a Multi-Step Process



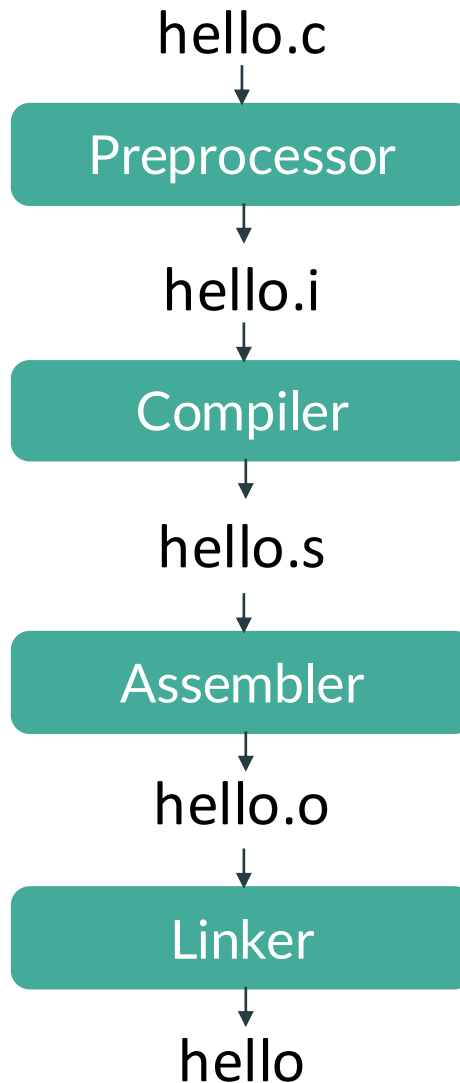
Compilation is a Multi-Step Process



The preprocessor handles directives such as `#include` before compilation.

Compilation is a Multi-Step Process

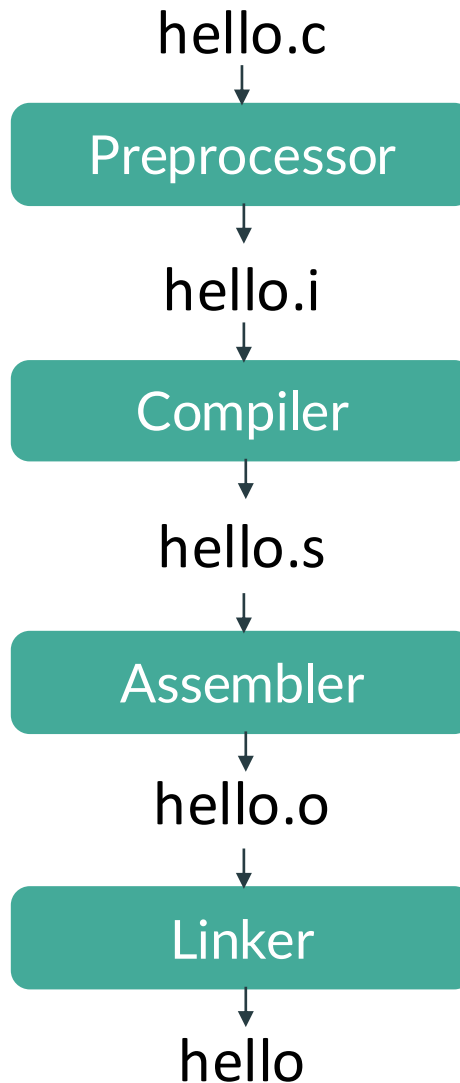
The compiler translates preprocessed c code into assembly language instructions.



The preprocessor handles directives such as `#include` before compilation.

Compilation is a Multi-Step Process

The compiler translates preprocessed c code into assembly language instructions.



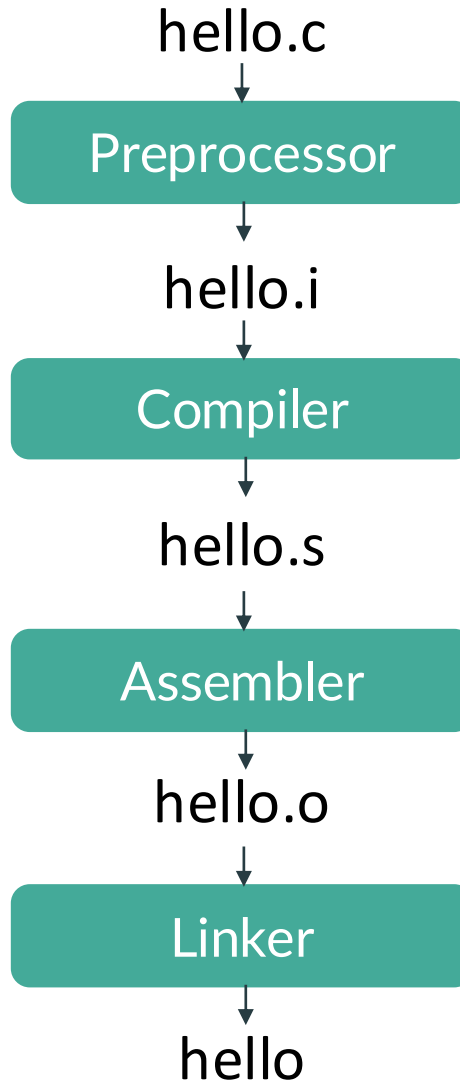
The preprocessor handles directives such as `#include` before compilation.

The assembler converts assembly code into machine code.

Compilation is a Multi-Step Process

The compiler translates preprocessed c code into assembly language instructions.

The linker combines one or more object files into a complete executable.



The preprocessor handles directives such as `#include` before compilation.

The assembler converts assembly code into machine code.

Separate Steps Reduce Work

If you have multiple programs that use the same source code:

- Compile each source file once and reuse the object file when building executables

If you have a lot of source files:

- Only compile the files that have changed since the last compilation

GCC Does All the Steps

To compile a program with gcc:

```
gcc [options] <c files>
```

Useful Options:

- -o specifies the output file name
- -g enables debugging
- -Wall checks for all warnings
- -lm links the math library

Preprocessor Directives

- File Inclusion
- Constants

#include Inserts the Contents of a File

#include copies the contents of a file into your file.

To include a standard library:

```
#include <stdio.h>
```

To include a developer file:

```
#include "utils.h"
```

#define Creates a Constant

#define creates a constant that is substituted everywhere in your file.

To define a constant:

```
#define MY_INT 10  
#define MY_STRING "hello"
```

#define Creates a Constant

#define creates a constant that is substituted everywhere in your file.

To define a constant:

```
#define MY_INT 10  
#define MY_STRING "hello"
```

Everywhere in your file:

MY_INT will be replaced by 10

MY_STRING will be replaced by "hello"

Make

- Make and Makefile
- Rules
- Variables
- Dependencies

Automate Compilation with Make

make is a program that reads the rules in a Makefile to tell it how to compile a program.

- Run complicated compilation by typing 'make'
- Only re-compile objects and executables that depend on files that have been changed

Makefiles Contain Rules

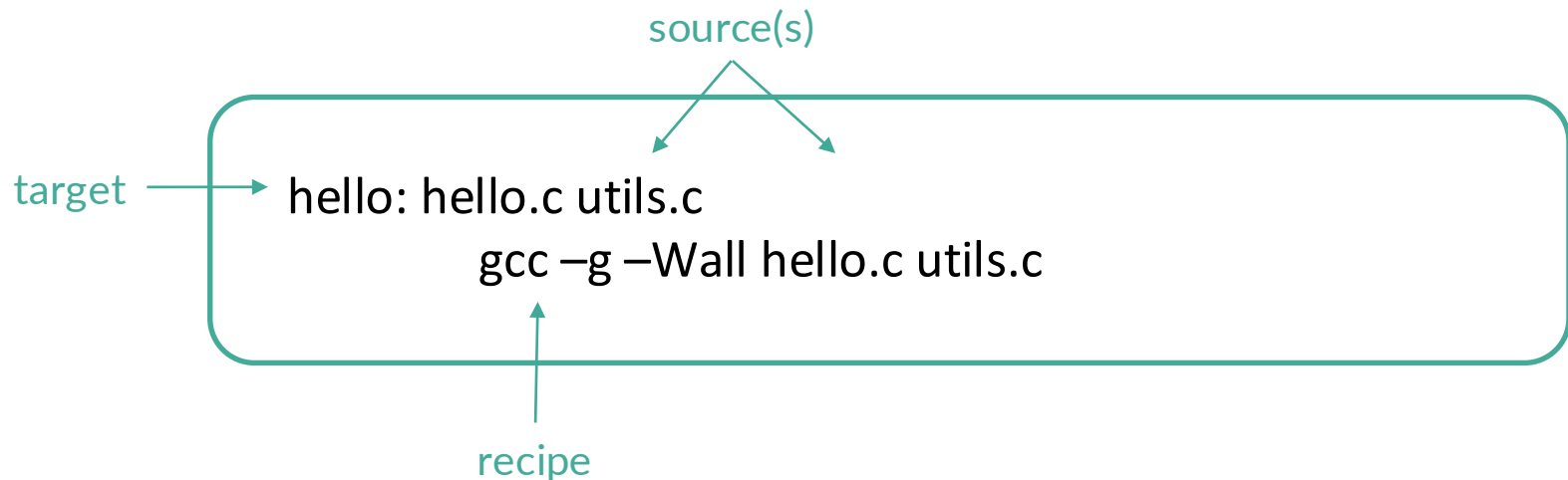
A **rule** consists of:

- A **target** – the output file to be generated
- One or more **sources** – input source code files the target depends on
- A **recipe** – list of commands to generate the target

Makefiles Contain Rules

A **rule** consists of:

- A **target** – the output file to be generated
- One or more **sources** – input source code files the target depends on
- A **recipe** – list of commands to generate the target



Make is Picky About Whitespace

```
hello: hello.c utils.c  
    gcc -g -Wall hello.c utils.c
```



This is a TAB,
spaces won't work

Make has Variables

Declare and use variables in your Makefile:

```
CC = gcc
```

```
CFLAGS = -g -Wall
```

```
hello: hello.c utils.c
```

```
    $(CC) $(CFLAGS) hello.c utils.c
```

Make Automatically Follows Dependencies

By default, make starts with the first rule in the file:

```
make
```

You can also specify one or more targets:

```
make [targets]
```

If a source is also a target for other sources, make will automatically follow those dependencies and run each rule needed to be able to run the original target.

Some Rules Don't Generate a File

Instead, they are used to run other useful commands.

```
all: target1 target2 test_suite
```

```
clean:
```

```
    rm target1 target2 test_suite
```

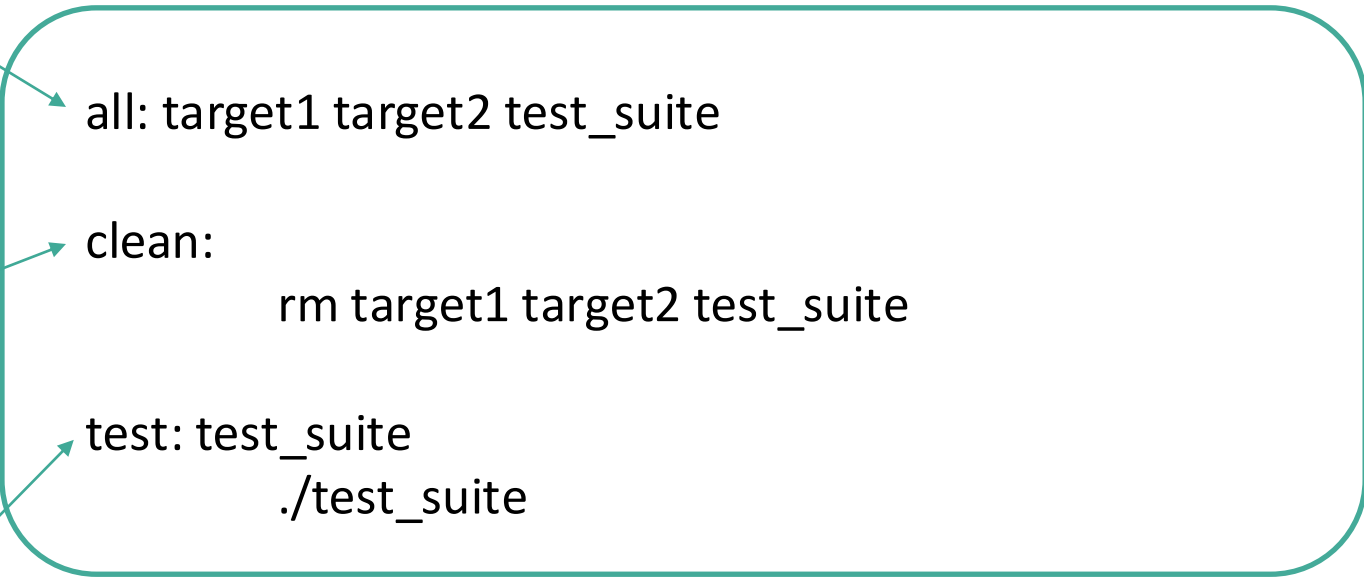
```
test: test_suite
```

```
    ./test_suite
```

Some Rules Don't Generate a File

Instead, they are used to run other useful commands.

All the targets for a complete build



```
all: target1 target2 test_suite
```

Remove
generated files

```
clean:
```

```
    rm target1 target2 test_suite
```

```
test: test_suite
```

```
    ./test_suite
```

Run a series of tests

Debugging With GDB

GDB Is A Symbolic Debugger

GDB lets you step through a program line by line and examine variables during program execution.

With GDB you can:

- Pause, resume, and step through program execution
- Observe variable values and memory locations
- Change variable values and memory locations

Step Through Code Line By Line

step runs the current line of code, and, if it's a function call, goes inside the function.

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Step Through Code Line By Line

step runs the current line of code, and, if it's a function call, goes inside the function.

(gdb) step



```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Step Through Code Line By Line

step runs the current line of code, and, if it's a function call, goes inside the function.

```
(gdb) step  
sum(x=1, y=7) at 12  
(gdb)
```

```
1 int sum(int x, int y);  
2  
3 int main(){  
4   int a = 1;  
5   int b = 7;  
6   int s = sum(a, b);  
7   printf("sum = %d", s);  
8   return 0;  
9 }  
10  
11 int sum(int x, int y){  
12   int sum = x+y;  
13   return sum;  
14 }
```



Step Through Code Line By Line

next runs the current line of code, and, if it's a function call, run the entire function.

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Step Through Code Line By Line

next runs the current line of code, and, if it's a function call, run the entire function.

(gdb) next

1 int sum(int x, int y);

2

3 int main(){

4 int a = 1;

5 int b = 7;



6 int s = sum(a, b);

7 printf("sum = %d", s);

8 return 0;

9 }

10

11 int sum(int x, int y){

12 int sum = x+y;

13 return sum;

14 }

Step Through Code Line By Line

next runs the current line of code, and, if it's a function call, run the entire function.

```
(gdb) next  
(gdb)
```

```
1 int sum(int x, int y);
```

```
2
```

```
3 int main(){
```

```
4   int a = 1;
```

```
5   int b = 7;
```

```
6   int s = sum(a, b);
```

```
➔ 7   printf("sum = %d", s);
```

```
8   return 0;
```

```
9 }
```

```
10
```

```
11 int sum(int x, int y){
```

```
12   int sum = x+y;
```

```
13   return sum;
```

```
14 }
```

Breakpoints Pause Execution

`break <line | func>` pauses execution at a specific line number or function.

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Breakpoints Pause Execution

break <line | func> pauses execution at a specific line number or function.

```
(gdb) break sum
(gdb) run
```

```
1 int sum(int x, int y);
2
3 int main(){
4   int a = 1;
5   int b = 7;
6   int s = sum(a, b);
7   printf("sum = %d", s);
8   return 0;
9 }
10
11 int sum(int x, int y){
12   int sum = x+y;
13   return sum;
14 }
```

Breakpoints Pause Execution

break <line | func> pauses execution at a specific line number or function.

```
(gdb) break sum
(gdb) run
sum(x=1, y=7) at 12
(gdb)
```

```
1 int sum(int x, int y);
2
3 int main(){
4     int a = 1;
5     int b = 7;
6     int s = sum(a, b);
7     printf("sum = %d", s);
8     return 0;
9 }
10
11 int sum(int x, int y){
12     int sum = x+y;
13     return sum;
14 }
```



Continue Resumes Execution

continue continues execution
until an error, the next
breakpoint, or execution ends.

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Continue Resumes Execution

continue continues execution until an error, the next breakpoint, or execution ends.

(gdb)continue

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```



Continue Resumes Execution

continue continues execution until an error, the next breakpoint, or execution ends.

```
(gdb)continue  
Continuing.  
sum = 8  
prog. exited normally  
(gdb)
```

```
1 int sum(int x, int y);  
2  
3 int main(){  
4     int a = 1;  
5     int b = 7;  
6     int s = sum(a, b);  
7     printf("sum = %d", s);  
8     return 0;  
9 }  
10  
11 int sum(int x, int y){  
12     int sum = x+y;  
13     return sum;  
14 }
```

Using GDB

1. Compile code for debugging

```
gcc -g file.c -o executable
```

2. Start GDB

```
gdb -tui ./executable
```

GDB Commands

<code>gdb -tui ./executable</code>	start GDB
<code>run</code>	run the program
<code>start</code>	run the program and stop at <code>main()</code>
<code>quit</code>	quit GDB
<code>step</code>	execute one line, going into function calls
<code>next</code>	execute one line, stepping over function calls
<code>continue</code>	continue execution
<code>finish</code>	finish current function call, then break
<code>break <line func></code>	set breakpoint

GDB Commands

list	display source code
print <var>	show variable value
display <var>	show variable every time execution stops
undisplay <var>	remove variable from list
watch <var>	stop execution whenever var changes
clear	remove breakpoints
delete breakpoint <n>	remove breakpoint n
backtrace	print the call stack
ctrl-c	stop program but stay in GDB

Demo: GDB

Exercise: GDB